

Calling Convention

Application Binary Interface (ABI)

- [*] How arguments are passed
- [*] How return values are returned
- [*] Which registers a function may destroy
- [*] Which registers must be preserved
- [*] Stack alignment rules
- [*] Binary format conventions

ABI Differences

[−] **AMD64 + SYSV**

[*] **args** → rdi, rsi, rdx, rcx, r8, r9

[*] **ret** → rax

[−] **x86**

[*] **args** → stack

[*] **ret** → eax

[−] **Aarch64 (ARM64)**

[*] **args** → x0-x7

[*] **ret** → x0

[−] **MIPS64**

[*] **args** → a0-a3

[*] **ret** → v0

[−] **RISC-V 64 (RV64)**

[*] **args** → a0-a7

[*] **ret** → a0

Prologue & Epilogue

```
void  
test()  
{  
    int x = 5;  
}
```

; amd64

test:

prologue

epilogue

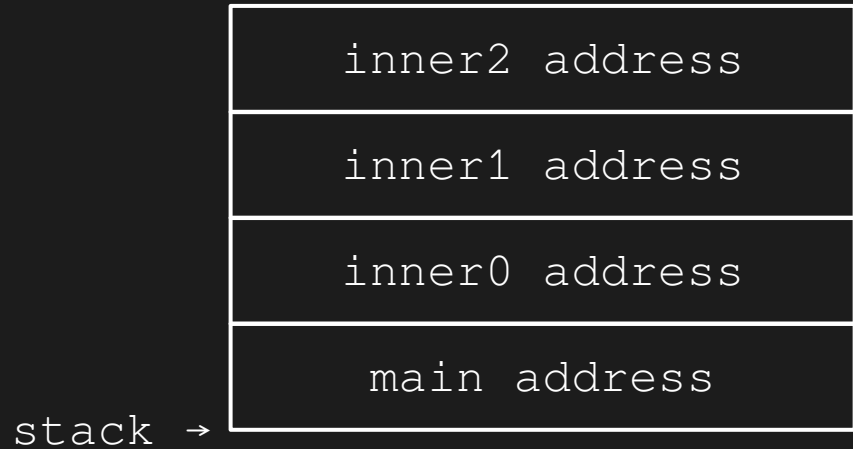
Nested Calls

```
void
inner2(void)
{
    return;
}

void
inner1(void)
{
    inner2();
}

void
inner0(void)
{
    inner1();
}

void
main(void)
{
    inner0();
}
```



* on **x86**, **call** would push the function address into the stack, where **ret** would pop the address

Interrupt Handlers

- [*] Hardware actions
- [*] Software anomalies
- [*] Stops the current work
- [*] Often at specific locations

```
# MIPS64
interrupt_handler:
    addiu sp, sp, -32

    sw ra, 28(sp)
    sw t0, 24(sp)
    sw t1, 20(sp)

    # handle interrupt

    lw t1, 20(sp)
    lw t0, 24(sp)
    lw ra, 28(sp)

    addiu sp, sp, 32

    eret
```

System Calls

- [*] Read arbitrary RAM
- [*] Access disks directly
- [*] Configure network interfaces
- [*] Control hardware
- [*] Disable memory protection

- [!] Modern OSes enforce **user** vs **kernel** space
- [!] Asks the kernel to perform the given task

Common Syscalls

```
read()  
write()  
open()  
close()  
fork()  
execve()  
mmap()  
munmap()  
socket()  
connect()  
...
```


The Workflow

```
printf("hello");
```

```
write(...);
```

```
syscall(...);
```

```
kernel(...);
```

```
terminal(...);
```

user application
libc
system call
linux kernel
hardware

AMD64 syscall

```
.section .data
msg:
    .ascii "Hello\n"
len = . - msg

.section .text
.global _start

_start:
    # write(1, msg, len)
    mov $1, %rax        # SYS_write
    mov $1, %rdi        # fd
    mov $msg, %rsi     # buffer
    mov $len, %rdx     # length

    syscall

    # exit(0)
    mov $60, %rax      # SYS_exit
    xor %rdi, %rdi

    syscall
```

ARM64 syscall

```
.section .data
msg:
    .ascii "Hello\n"
len = . - msg

.section .text
.global _start

_start:
    # write(1,msg,len)
    mov x0, #1
    ldr x1, =msg
    mov x2, #len
    mov x8, #64          // SYS_write

    svc #0

    # exit(0)
    mov x0, #0
    mov x8, #93          // SYS_exit

    svc #0
```

MIPS64 syscall

```
.data
msg:
    .ascii "Hello\n"

.text
.globl _start
_start:
    # write(1,msg,6)
    li      $a0, 1
    la      $a1, msg
    li      $a2, 6
    li      $v0, 5001        # __NR_write

    syscall

    # exit(0)
    move    $a0, $zero
    li      $v0, 5058        # __NR_exit

    syscall
```

RISC64 syscall

```
.section .data
msg:
    .ascii "Hello\n"
len = . - msg

.section .text
.global _start
_start:
    # write(1,msg,len)
    li a0, 1
    la a1, msg
    li a2, len
    li a7, 64          # SYS_write

    ecall

    # exit(0)
    li a0, 0
    li a7, 93          # SYS_exit

    ecall
```

Questions?